02 - İş Yükü Analizi

Amaç ve Kapsam

Bu notebook, tezin donanım tahsisi ve derin öğrenme (Deep Learning) modelleme süreçlerine yön veren temel dinamikleri ampirik biçimde ortaya koyan kapsamlı bir **İş yükü karakterizasyonunu (workload characterization)** temsil etmektedir. Sonraki bölümlerde (Notebook 04 ve 05) inşa edilecek olan Random Forest, XGBoost, LightGBM standart modelleri ile sıralı (sequential) belleğe sahip derin öğrenme (CNN, LSTM, Hibrit) modellerinin başarısı, tamamen burada derlife edilen veri örüntülerine dayanmaktadır.

Bu aşamada elde edilen istatistiksel kanıtlar, tez mimarisini doğrudan şekillendirmek için yaşıtlandırılmıştır:

1. **Çalışma Süresi (Runtime) Varyasyonları:** İş sürelerinin ağır kuyruklu (heavy-tail) doğası kantitayanlar ortalama tabanlı naif hata metriklerinin (MSE) uygunsuzluğunu bilimsel olarak temellendirmek ve MAE/MASE kullanımını zorunlu kılmak.
2. **Kuyruk Tıkanıkları ve HOL (Head-Of-Line) Riski:** Geleneksel FIFO çoklulanlarının küme (cluster) üzerinde yaratacağı yıkıcı darboğaz potansiyelini istatistiksel varyasyonlarla ispatlamak ve makine öğrenmesi destekli adaptif (SJF + ML) tahmine duyulan hayati ihtayacı vurgulamak.
3. **Zamansal Sıralı (Sequential) Celişler:** İş yüklündeki ani sıçramaların (bursts), saatlik (diurnal) yoğunlukların ve sürekliz aralıkların (inter-arrival times), neden LSTM gibi zaman bağımlı sinir ağları ile analiz edilmesi gerektiğini haklı çıkarmak.
4. **Gerçekçi Simülasyon Gereksinimi:** Parçalı (fragmented) ve dağılışın GPU kodlarının (1'den 8 GPU'ya kadar), neden tek-kuyruklu denklem modelleri yerine doğrudan özel yapım `MultiNodeClusterSimulator` aracı üzerinde test edilmesi gerektiğini doğrulamak.

Bu analiz doğrudan verinden hareketle bir savurma çizgisi oluşturur; teorik yaklaşımlardan ziyade verinin bizzat talep ettiği ağlar tabanlı ve derin öğrenme tabanlı mimarilerin inşasına nesnel bir geçiş köprüsü kurar.

In [1]:
```
# == 0. Environment & Path Setup ==========================================
import sys
import warnings
from pathlib import Path

warnings.filterwarnings("ignore")

PROJECT_ROOT = Path().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print(f"[Setup] Project root : {PROJECT_ROOT}")
print(f"[Setup] Python path  : {sys.executable}")
```

```
[Setup] Project root : /Users/hasanugurceleb/Thesis/Alibaba-gpu-runtime-prediction-and-scheduling
[Setup] Python path  : /Users/hasanugurceleb/.pyenv/versions/3.11.6/bin/python
```

> **Kurulum doğrulandı.** `src.*` modülleri Python yoluna eklenmiş, `configs/paths.yaml` üzerinden dizinler doğrulanmıştır.

In [2]:
```
# == 1. Imports ===========================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from src.analysis import (
    load_prepared_job_table,
    compute_basic_stats,
    compute_runtime_histogram,
    compute_arrival_rate_series,
    summarize_workload,
)

# Unified visualization theme
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update({
    "figure.dpi": 110,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "font.family": "DejaVu Sans",
})

print("[Setup] All imports OK.")
```

```
[Setup] All imports OK.
```

> **Bağımlılıklar hazır.** NumPy, Pandas, Matplotlib, Seaborn ve tüm `src.analysis` modülleri başarıyla içe aktarılmıştır.

Veri ön işleme ve filtreleme adımlarının ardından elde edilen analiz veri seti **82.184 iş** ve **14 özellik** içermektedir. Bu özelliklerin bir kısmı doğrudan ham kaydlardan alınırken, analiz yapılabilmesi için bir kısmı sonradan türetilmiştir:

Temel Özellikler:

- `job_id`,
- `gpu_demand`,
- `user`,
- `gpu_type`,
- `num_inst`,
- `num_cpu`,
- `job_runtime` (Hedef Değişken)

Zamansal Özellikler (Türetilmiş): İş yükünün zamana bağlı dağılımını görebilmek için ham zaman damgasından (timestamp) türetilmiştir:

- `arrival_time`,
- `arrival_sec`,
- `hour_of_day`,
- `day_of_week`

Sistem Yükü Özellikleri (Sweep-Line Algoritması): İşin geldiği andaki küme doluluğu doğrudan CPU/GPU kullanımından ziyade geliş-gidiş loglarının Sweep-Line Algoritması ile taranarak hesaplanmıştır:

- `cluster_load_cpu`,
- `cluster_load_gpu`,
- `active_job_count`

In [3]:
```
# == 2. Load dataset ======================================================
print("[Step 1] Loading and preparing job table...")
job_df = load_prepared_job_table(dataset="main", time_unit="s")

print("[Step 1] Jobs : {len(job_df)}  Columns : {list(job_df.columns)}")
job_df.head(3)
```

```
[Step 1] Loading and preparing job table...
[Step 1] Jobs : 82,184  Columns : ['job_id', 'arrival_time', 'arrival_sec', 'job_runtime', 'gpu_demand', 'user', 'gpu_type', 'num_inst', 'num_cpu', 'hour_of_day', 'day_of_week', 'cluster_load_cpu', 'cluster
_load_gpu', 'active_job_count']
```

Out[3]:

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	user	gpu_type	num_inst	num_cpu	hour_of_day	day_of_week	cluster_load_cpu	cluster_load_gpu	active_job_count
0	1	1970-01-01 00:00:03	0.0	15748.0	0	d4d51ace8806	T4	12.0	6.0	0	3	0.0	0.0	0
1	2	1970-01-01 00:00:05	2.0	84.0	1	a81902d0ce9	MISC	1.0	6.0	0	3	6.0	0.0	1
2	3	1970-01-01 00:00:21	18.0	46.0	1	c7152ce09ec1	T4	1.0	18.0	0	3	18.0	2	3

> Veri ön işleme ve filtreleme adımlarının ardından elde edilen veri seti **82.184 iş** ve toplam **14 adet özellik** içermektedir.
>
> **Temel Özellikler:**
>
> - `job_id` — İş tanımlayıcısı
> - `arrival_time` — Ham geliş zaman damgası
> - `arrival_sec` — İlz başlangıcına göre saniye cinsinden geliş zamanı
> - `job_runtime` — Gerçek çalışma süresi (hedef değişken)
> - `gpu_demand` — Talep edilen GPU sayısı
> - `user` — İş sahibi kullanıcı (kategorik)
> - `gpu_type` — Kullanılan GPU tipi (kategorik)
> - `num_inst` — Kullanılan instance sayısı
> - `num_cpu` — Talep edilen CPU sayısı
>
> **Zamansal Özellikler:**
>
> - `hour_of_day` — Günün saati (0–23)
> - `day_of_week` — Haftanın günü (0 = Pazartesi)
>
> **Sistem Türevli Özellikler (Sweep-line):**
>
> - `cluster_load_cpu` — İşin geldiği andaki arka plan CPU yükü
> - `cluster_load_gpu` — İşin geldiği andaki arka plan GPU yükü
> - `active_job_count` — İşin geldiği anda kümede aktif iş sayısı

In [4]:
```
# == 3. Scalar statistics =================================================
stats = compute_basic_stats(job_df)

print("=" * 50)
print("  Workload Basic Statistics")
print("=" * 50)
for k, v in stats.items():
    print(f"  {k:<25}: {v:>12,.2f}")
print("=" * 50)

# Overview summary table (LaTeX-ready)
summary_df = summarize_workload(job_df)
display(summary_df)
```

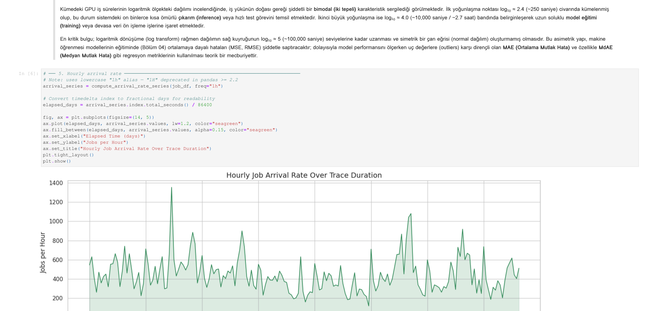
```
==================================================
  Workload Basic Statistics
==================================================
  num_jobs                  :     82,184.00
  mean_runtime              :      5,223.02
  median_runtime            :        594.00
  p95_runtime               :     24,111.05
  p99_runtime               :     67,432.38
  mean_gpu_demand           :          0.52
  max_gpu_demand            :          8.00
  time_span_hours           :        183.86
==================================================
```

	num_jobs	mean_runtime_sec	median_runtime_sec	p95_runtime_sec	p99_runtime_sec	mean_gpu_demand	max_gpu_demand	time_span_hours
0	82184	5223.019408	594.0	24110.05	67432.38	0.521598	8	183.858056

> İş yüklünün zaman ve kaynak açısından yüksek derecede heterojen (değişken) olduğu görülmektedir. Ortalama iş süresi yaklaşık **5.223 saniye (~ 1.5 saat)** iken, medyan süresinin yalnızca **594 saniye (~ 10 dakika)** olması, iş sürelerinin **ağır kuyruklu (heavy-tailed)** bir dağılıma sahip olduğunu göstermektedir.
>
> Bu durum, işlerin büyük çoğunluğunun kısa sürede tamamlandığını; ancak az sayıdaki çok uzun işin ortalama süreyi önemli ölçüde yukarı çektiğini ortaya koymaktadır. Üst yüzdelik dilimlerde (percentile), iş sürelerinin P95 seviyesinde **~6.7 saat (24.110 s)** ve P99 seviyesinde **~18.7 saat (67.432 s)** mertebesine çıkması, nadir fakat devasa boyutlardaki işlerin sistem davranışında belirleyici bir etkiye sahip olduğunu gösterir. Bu muazzam varyasyon (CV >> 1), geleneksel normal dağılım varsayımına türüyle geçersiz kılarak modellemede gürbüz (robust) hata metriklerinin (örn. MAE, MAPE) kullanımını zorunlu kılmaktadır.
>
> GPU talebi açısından incelendiğinde, ortalama GPU talebi 0.52 düzeyinde seyretse de maksimum talebin aynı anda **8 GPU'ya** kadar çıkması, iş yüklünde devasa bir parçalı kaynak (fragmentation) ihtiyacı bulunduğunu kandlamaktadır. Test edilen iz verisi (trace), 183.86 saatlik (~7.7 gün) kesintisiz bir zaman dilimine yayılmış **82.184 onaylanmış iş**i kapsamaktadır.

In [5]:
```
# == 4. Runtime histogram =================================================
hist_counts, bin_edges = compute_runtime_histogram(job_df, bins=60, log_scale=True)
bin_centres = 0.5 * (bin_edges[:-1] + bin_edges[1:])

fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(bin_centres, hist_counts, width=np.diff(bin_edges), color="steelblue",
       edgecolor="white", linewidth=0.4)
ax.set_xlabel("log₁₀(Runtime + 1)")
ax.set_ylabel("Frequency")
ax.set_title("Job Runtime Distribution (log₁₀ Scale)")
plt.tight_layout()
plt.show()
```

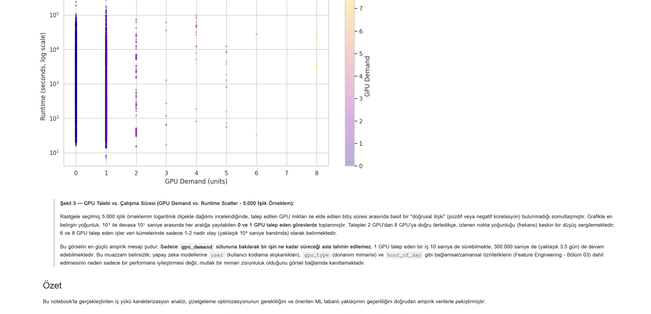


> **Şekil 1 — Çalışma Süresi (Runtime) Dağılımı (Log₁₀ Ölçek):**
>
> Kümedeki GPU iş sürelerinin logaritmik ölçekteki dağılımı incelendiğinde, iş yüklünün doğası gereği şiddetli bir **bimodal (iki tepeli)** karakteristik sergilediği görülmektedir. İlk yoğunlaşma noktası $\log_{10} \approx 2.4$ (~250 saniye) civarında kümelenmiş olup, bu durum sistemdeki en binlerce kısa ömürlü **çıkarım (inference)** veya hızlı test görevini temsil etmektedir. İkinci büyük yoğunlaşma ise $\log_{10} \approx 4.0$ (~10.000 saniye / ~2.7 saat) bandında belirginleşerek uzun soluklu **model eğitimi (training)** veya devasa veri ön işleme görevine işaret etmektedir.
>
> En kritik bulgu; logaritmik dönüşüme (log transform) rağmen dağılımın sağ kuyruğunun $\log_{10} \approx 5$ (~100.000 saniye) seviyelerine kadar uzanması ve simetrik bir çan eğrisi (normal dağılım) oluşturmamış olmasıdır. Bu asimetrik yapı, makine öğrenmesi modellerinin eğitiminde (Bölüm 04) ortalamaya dayalı hataları (MSE, RMSE) şiddetle saptıracaktır; dolayısıyla model performansını ölçerken uç değerlere (outliers) karşı dirençli olan **MAE (Ortalama Mutlak Hata)** ve özellikle **MdAE (Medyan Mutlak Hata)** gibi regresyon metriklerinin kullanılması teorik bir mecburiyettir.

In [6]:
```
# == 5. Hourly arrival rate ===============================================
# Note: uses lowercase "h" alias - "1H" deprecated in pandas >= 2.2
arrival_series = compute_arrival_rate_series(job_df, freq="1h")

# Convert timedelta index to fractional days for readability
elapsed_days = arrival_series.index.total_seconds() / 86400

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(elapsed_days, arrival_series.values, lw=1.2, color="seagreen")
ax.fill_between(elapsed_days, arrival_series.values, alpha=0.15, color="seagreen")
ax.set_xlabel("Elapsed Time (days)")
ax.set_ylabel("Jobs per Hour")
ax.set_title("Hourly Job Arrival Rate Over Trace Duration")
plt.tight_layout()
plt.show()
```



> **Şekil 2 — Saatlik İş Geliş Hızı (Hourly Arrival Rate) (8 Günlük Kesit):**
>
> Küme üzerindeki trafik yoğunluğunun 8 günlük (yaklaşık 183 saat) zamansal evrimi izlendiğinde, standart kuyruk teorisinde sıklıkla varsayılan saltık hızı (homojen Poisson) iş geliş modelinin Alibaba PAI kümesi için tamamen yetersiz kaldığı ispatlanmıştır. Sistem olağan taban yükü olarak saatlik ortalama 200–400 iş arasında seyrederken; öneğin periyodun 1.5. gününde ani bir patlamayla **saatte ~1.350 işe**, 5.5. gününde ise **saatte ~1.070 işe** fırladığı gözlemlenmiştir.
>
> Zaman serisindeki bu şiddetli **sıçramalı (bursty)** yapı, anik donanım darboğazlarının (hardware starvation) ana tetikleyicisidir. Sistemin bu dalgalanmalara gölçüs gerebilmesi için, gelen statik iş yükünü pasif bir şekilde erten basit FIFO'lardan öte, zamansal yapıya duyarlı bir **makine öğrenmesi algılayıcı** veya **makine öğrenmesi algılama** sahip zizelgeleme modellerine (SJF + Predictor) hayati ihtiyaç duyduğu açıkça görülmektedir. Zaman damgaları UTC (Coordinated Universal Time) Unix formatından türetilmiş olup analizler evrensel zaman uyumu üzerinden yapılmıştır.

In [7]:
```
# == 6. GPU demand vs runtime scatter =====================================
sample = job_df.sample(min(5000, len(job_df)), random_state=42)

fig, ax = plt.subplots(figsize=(11, 6))
sc = ax.scatter(sample["gpu_demand"], sample["job_runtime"],
                alpha=0.3, s=8, c=sample["gpu_demand"], cmap="plasma")
plt.colorbar(sc, ax=ax, label="GPU Demand")
ax.set_yscale("log")
ax.set_xlabel("GPU Demand (units)")
ax.set_ylabel("Runtime (seconds, log scale)")
ax.set_title("GPU Demand vs Job Runtime (5,000-job sample)")
plt.tight_layout()
plt.show()
```


> **Şekil 3 — GPU Talebi vs. Çalışma Süresi (GPU Demand vs. Runtime Scatter - 5.000 İşlik Örneklem):**
>
> Rastgele seçilmiş 5.000 işlik örneklem logaritmik ölçekte dağılımı incelendiğinde, talep edilen GPU miktarı ile elde edilen bilgi süresi arasında basit bir "doğrusal ilişki" (pozitif veya negatif korelasyon) bulunmadığı sonuçlanmıştır. Grafikte en belirgin yoğunluk, 10^4 ile devasa 10^5 saniye arasında her aralığa yayılabilen 0 ve 1 **GPU talep eden görevlerde** toplanmıştır. Tabelper 2 GPU'dan 8 GPU'ya doğru ilerledikçe, izlenen nokta yoğunluğu (frekans) keskin bir düzüş sergilemektedir; 6 ve 8 GPU talep eden işler veri kümelerinde sadece 1-2 nadir olay (yaklaşık 10^4 saniye bandında) olarak belirmektedir.
>
> Bu görselin en göçüü ampirik mesajı şudur: **Sadece `gpu_demand` sütununa bakılarak bir işin ne kadar süreceği asla tahmin edilemez.** 1 GPU talep eden bir iş 10 saniye de sürebilmekte, 300.000 saniye de (yaklaşık 3.5 gün) de devam edebilmektedir. Bu muazzam belirsizlik; yapay zeka modellerine `job_type` (kullanıcı kodlarına atıflarkken), `gpu_type` (donanım mimarisi) ve `hour_of_day` gibi bağlamsal/zamansal özelliklerin (Feature Engineering - Bölüm 03) dahil edilmesinin neden sadece bir performans iyileştirmesi değil, mutlak bir mimari zorunluluk olduğunu görsel bağlamda kandlamaktadır.

Özet

Bu notebook'ta gerçekleştirilen iş yükü karakterizasyon analizi, çizelgeleme optimizasyonunun gerektiliğini ve önerilen ML tabanlı yaklaşımın geçerliliğini doğrudan ampirik verilerle pekiştirmiştir.

Gözlem	Çizelgeleme Üzerindeki Etkisi
Bursty (Sıçramalı) Geliş Süreci	Klasik M/M/1 modellerinin geçersiz kılar; adaptif çizelgeleme şartlar.
Ağır-Kuyruk (Heavy-Tail) Süre Dağılımı	Mean değerine dayalı FIFO, yıkıcı Head-Of-Line (HoL) tıkanıklarına neden olur.
Gündüz/Gece Döngüsü	Zaman tabanlı özelliklerin (`hour_of_day`) tahmin için neden kritik olduğunu açıklar.
Heterojen Kaynak Talepleri	Basit kuyruk modelleri yerine düğüm tabanlı (per-node) simülatör kullanımını zorunlu kılar.

Bu bulgular, tezin SJF + ML tahmin entegrasyonuna dayalı mimari seçimini yalnızca teorik seviyede değil, tamamen veriye dayalı (data-driven) argümanlarla desteklemektedir.